

T.C. MALTEPE UNIVERSITY
FACULTY OF ENGINEERING AND NATURAL SCIENCES

SE 308 01 Advanced Database Systems
Term Project Document

Author:

İpek Dedeoğlu
220706043

Instructor:

Volkan Tunalı

1. Introduction

This project focuses on simulating concurrent transactions in a relational database system using different isolation levels. The purpose is to observe how transaction performance and deadlocks are affected under varying loads and isolation levels in SQL Server. The AdventureWorks sample database is used for experimentation.

We created a Windows Forms simulation application where we can dynamically select the number of Type A (update) and Type B (select) users, define the isolation level, and measure execution time and deadlocks encountered.

2. System Architecture

The project is implemented as a Windows Forms application using C# and SQL Server. Key components include:

- **UI Controls:**
 - NumericUpDown controls for number of Type A/B users.
 - ComboBox for isolation level selection.
 - Button to start the simulation.
 - ProgressBar and TextBox for visual feedback.
- **Backend Logic:**
 - Threads created for each user, handling their respective transaction logic.
 - Logging and timing handled per thread.
 - Deadlock exceptions caught and logged without interrupting simulation.

3. Simulation Design

Type A Users:

- Perform update transactions 100 times.
- Each transaction connects to the database, runs conditional updates, and disconnects.

Type B Users:

- Perform select transactions 100 times.
 - Each transaction connects, reads data with consistent select queries, and disconnects.
- Each user operates independently in its own thread. Randomization is used to make simulation more realistic and mimic real-world concurrent systems.

4. Application Development

❖ **Development Environment**

The application was developed using the following technologies:

- Development Environment: Microsoft Visual Studio
- Programming Language: C# • Database Management System: Microsoft SQL Server Management Studio 2022
- Database: AdventureWorks2022

❖ **User Interface**

The user interface allows to choose number of Type A and Type B user, isolation level.

After running the program by clicking Start Simulation Button user can see the values.

❖ **Form Page:**

The screenshot shows a Windows application window titled "Form1". At the top, there are two spin boxes: "Type A" with a value of 0 and "Type B" with a value of 0. Below them is a label "Isolation Level" followed by a dropdown menu showing "READ UNCOMMITTED". In the center is a large table with 7 columns: "Thread ID", "User Type", "Step", "Status", "Execution Time", and "Deadlock". The table has a header row and one data row containing an asterisk (*). At the bottom right of the form is a button labeled "Start Simulation".

	Thread ID	User Type	Step	Status	Execution Time	Deadlock
*						

❖ **Result Page:**

The screenshot shows the same Windows application window "Form1" but with updated values. The "Type A" spin box now shows 65 and "Type B" shows 80. The "Isolation Level" dropdown remains "READ UNCOMMITTED". The table now contains 17 rows of data. The first row is highlighted in blue and has a play button icon in the first column. The columns are: Thread ID, User Type, Step, Status, Execution Time, and Deadlock. At the bottom right is a button labeled "Start Simulation".

	Thread ID	User Type	Step	Status	Execution Time	Deadlock
▶	SYSTEM	-	-	INFO	Simulation Starte...	-
	B2	B	1	OK	-	NO
	B8	B	1	OK	-	NO
	B6	B	1	OK	-	NO
	B1	B	1	OK	-	NO
	B3	B	1	OK	-	NO
	B0	B	1	OK	-	NO
	B5	B	1	OK	-	NO
	A0	A	1	OK	-	NO
	A2	A	1	OK	-	NO
	B10	B	1	OK	-	NO
	B17	B	1	OK	-	NO
	B16	B	1	OK	-	NO
	B9	B	1	OK	-	NO

5. Experiment Setup

- **Isolation Levels Tested:**

- **READ COMMITTED:** Significant increases in both transaction durations and deadlocks as the number of users grows. Performance is noticeably slower, and deadlocks are more frequent compared to indexed scenarios. While Read Committed provides a balance between performance and data consistency, the lack of indexing leads to higher resource usage and contention issues, especially under high concurrency.
- **READ UNCOMMITTED:** As the number of users increases, the average duration for Type threads increases , and the number of deadlocks encountered by Type A users also rises massively. Type B threads maintain a low and stable duration with no deadlocks. Type A transactions are more likely to encounter deadlocks as concurrency increases, while Type B transactions face no change.
- **REPEATABLE READ:** Both Type A and Type B threads experience increasing durations and deadlocks with higher concurrency. Type B users face significant deadlocks compared to READ UNCOMMITTED. READ COMMITTED introduces higher contention and deadlocks for both Type A and Type B users, indicating more robust isolation but higher performance costs.
- **SERIALIZABLE:** Highest average duration and deadlocks for both Type A and Type B users, especially with high concurrency. SERIALIZABLE provides the highest level of isolation but with lower and weaker performance , making it impractical for highconcurrency scenarios.

With no indexes:

Expectations ->

- Slower query execution.
- Higher number of deadlocks due to full table scans.

With indexes:

Expectations ->

- Improved performance.
- Fewer deadlocks.

Metrics Measured:

- Average execution time per thread (Type A and B).
- Number of deadlocks encountered by each type.

6. Results

✓ Indexed Results

Indexed Results READ UNCOMMITTED

Number of Users Type A	Average Duration of Type A Users	Number of Deadlock Encountered by Type A	Number of Users Type B	Average Duration of Type B Users	Number of Deadlock Encountered by Type B
100	2957047,7195 ms	13	100	102804,9814 ms	0
65	2871613,5475 ms	0	80	65622,7694 ms	0

Indexed Results READ COMMITTED

Number of Users Type A	Average Duration of Type A Users	Number of Deadlock Encountered by Type A	Number of Users Type B	Average Duration of Type B Users	Number of Deadlock Encountered by Type B
95	2923075,9442 ms	0	98	126480,3215 ms	0
50	2788897,3512 ms	0	66	83216,2105 ms	0

**Indexed Results
REPEATABLE READ**

Number of Users Type A	Average Duration of Type A Users	Number of Deadlock Encountered by Type A	Number of Users Type B	Average Duration of Type B Users	Number of Deadlock Encountered by Type B
99	2838014,5897 ms	1524	99	1264909,1265 ms	58
50	2688131,9243 ms	1006	66	766158,8025 ms	74

**Indexed Results
SERIALIZABLE**

Number of Users Type A	Average Duration of Type A Users	Number of Deadlock Encountered by Type A	Number of Users Type B	Average Duration of Type B Users	Number of Deadlock Encountered by Type B
97	2729979,0522 ms	1399	100	1878458,8816 ms	96
61	1662284,0291 ms	3602	46	653777,0156 ms	124

✓ **Non-Indexed Results**

**Non-Indexed Results
READ UNCOMMITTED**

Number of Users Type A	Average Duration of Type A Users	Number of Deadlock Encountered by Type A	Number of Users Type B	Average Duration of Type B Users	Number of Deadlock Encountered by Type B
100	2998262,2557 ms	892	100	115361,53 ms	0
65	2865414,424 ms	1558	80	63408,4258 ms	0

**Non-Indexed Results
READ COMMITTED**

Number of Users Type A	Average Duration of Type A Users	Number of Deadlock Encountered by Type A	Number of Users Type B	Average Duration of Type B Users	Number of Deadlock Encountered by Type B
95	2918940,2063 ms	1237	98	198035,0461 ms	0
50	2879645,2237 ms	164	66	88869,0251 ms	0

**Non-Indexed Results
REPEATABLE READ**

Number of Users Type A	Average Duration of Type A Users	Number of Deadlock Encountered by Type A	Number of Users Type B	Average Duration of Type B Users	Number of Deadlock Encountered by Type B
99	2883633,7977 ms	898	99	1284257,6014 ms	6
48	2516434,0048 ms	1577	59	717743,3868 ms	108

**Non-Indexed Results
SERIALIZABLE**

Number of Users Type A	Average Duration of Type A Users	Number of Deadlock Encountered by Type A	Number of Users Type B	Average Duration of Type B Users	Number of Deadlock Encountered by Type B
97	2855093,8916 ms	899	100	1802888,075 ms	118
61	1262238,9105 ms	4642	46	597708,5739 ms	184

7. Analysis

Analysis of Indexed vs Non-Indexed Results Across Isolation Levels

1. READ UNCOMMITTED

❖ Indexed vs Non-Indexed – Comparison within Read Uncommitted

Under indexed conditions, both Type A and Type B users performed slightly better in terms of average transaction durations. More importantly, the number of deadlocks for Type A users dropped drastically when indexing was applied.

- For higher user counts (100 Type A, 100 Type B), deadlocks in the non-indexed setup reached 892, while the indexed version only encountered 13 deadlocks.
- At the lower user configuration (65 Type A, 80 Type B), the non-indexed scenario resulted in 1,558 deadlocks, but with indexing, it was completely eliminated (0).
This highlights how indexing dramatically reduces lock contention and deadlocks even in the lowest isolation level.

❖ Conclusion

Although Read Uncommitted allows for dirty reads and is inherently less strict with locking, indexing still plays a significant role in optimizing **write operations** and avoiding transaction collisions. The improvements in Type A user performance are the most striking outcome here.

2. READ COMMITTED

❖ Indexed vs Non-Indexed – Comparison within Read Committed

In the Read Committed level, indexing again proves highly effective in reducing deadlocks:

- In the high-load scenario (95 Type A, 98 Type B), the non-indexed setup experienced 1,237 deadlocks for Type A, while the indexed setup experienced none.
- In the mid-load scenario (50 Type A, 66 Type B), deadlocks dropped from 164 (non-indexed) to 0 (indexed).

Type B users in both setups never experienced deadlocks, but they showed better performance with indexing, indicating that optimized indexing can improve read query efficiency even in moderately strict isolation levels.

❖ Conclusion

Indexing is highly beneficial in Read Committed settings, particularly for write-heavy operations. It helps in avoiding blocking and improves read query times slightly. In systems where Read Committed is the default level, maintaining proper indexing can significantly enhance reliability and performance.

3. REPEATABLE READ

❖ Indexed vs Non-Indexed – Comparison within Repeatable Read

Unlike previous levels, Repeatable Read showed more mixed results:

- For high user counts (99 A/B users), deadlocks were higher in the indexed setup (1,524) compared to the non-indexed one (898).
- In the lower configuration (50 Type A, 66 Type B), deadlocks for Type A users were also slightly higher in the indexed setup (1,006 vs. 898).
- Type B user deadlocks also increased under indexing (58 → 74 at high load, 6 → 108 at lower load).

This pattern suggests that the stricter locking mechanisms in Repeatable Read, combined with indexing, may actually create more contention due to range locks and longer lock durations. Indexing doesn't always lead to better concurrency here.

❖ Conclusion

Indexing does not guarantee better performance in Repeatable Read. On the contrary, it may introduce more lock contention depending on the structure of queries and indexing strategies used. This isolation level requires careful index design and perhaps even changes in query logic to reduce concurrency issues.

4. SERIALIZABLE

❖ Indexed vs Non-Indexed – Comparison within Serializable

Serializable, being the strictest isolation level, showed **complex behavior**:

- For higher load (97 Type A, 100 Type B), Type A deadlocks increased with indexing (899 → 1,399), while Type B user deadlocks slightly improved (118 → 96).
- At the lower configuration (61 A, 46 B), indexing reduced Type A deadlocks (4,642 → 3,602) and Type B deadlocks (184 → 124).

So, the effect of indexing depends heavily on load conditions. Under heavy load, stricter isolation combined with indexing can increase the frequency of deadlocks, likely due to longer lock chains caused by the need to serialize access to resources. Under moderate load, however, indexing helps reduce lock contention and improves stability.

❖ Conclusion

Indexing in Serializable isolation level should be applied with great caution. It may lead to increased deadlocks if the system handles many simultaneous updates or range queries. However, it still offers potential performance benefits when the system is not overwhelmed with concurrent transactions.

Overall Insights

- Indexing greatly improves performance and reliability in lower isolation levels (Read Uncommitted, Read Committed), especially by preventing deadlocks for write-heavy operations.
- In higher isolation levels (Repeatable Read, Serializable), indexing needs to be used with a strategic design in mind. It doesn't always improve deadlocks and may even worsen contention due to more aggressive locking behaviors.
- Type A users (update-heavy) benefit the most from indexing in general, especially in scenarios with high concurrency.
- Type B users (read-heavy) show fewer issues with deadlocks overall, but indexing still helps to improve query durations.

8. Screenshots

The screenshot shows the Database Simulator application in Visual Studio. The 'Form1' window is open, displaying the configuration for the simulation. 'Type A' is set to 97, 'Type B' is set to 100, and the 'Isolation Level' is set to 'SERIALIZABLE'. The 'Start Simulation' button is at the bottom right. The background shows the Visual Studio code editor with the Database Simulator project files.

Thread ID	User Type	Step	Status	Execution Time	Deadlock
A0	A	1	ERROR	Transaction (Pro...	YES
A0	A	2	OK	-	NO
B2	B	1	ERROR	Transaction (Pro...	YES
B3	B	1	ERROR	Transaction (Pro...	YES
A1	A	1	ERROR	Transaction (Pro...	YES
B1	B	1	ERROR	Transaction (Pro...	YES
B20	B	1	ERROR	Transaction (Pro...	YES
A4	A	1	ERROR	Transaction (Pro...	YES
B5	B	1	ERROR	Transaction (Pro...	YES
B35	B	1	ERROR	Transaction (Pro...	YES
A5	A	1	ERROR	Transaction (Pro...	YES
A5	A	2	OK	-	NO
B4	B	1	ERROR	Transaction (Pro...	YES
B38	B	1	ERROR	Transaction (Pro...	YES

The screenshot shows the Database Simulator application in Visual Studio. The 'Form1' window is open, displaying the configuration for the simulation. 'Type A' is set to 95, 'Type B' is set to 98, and the 'Isolation Level' is set to 'READ COMMITTED'. The 'Start Simulation' button is at the bottom right. The background shows the Visual Studio code editor with the Database Simulator project files.

Thread ID	User Type	Step	Status	Execution Time	Deadlock
SYSTEM	-	-	INFO	Simulation State...	-
A0	A	1	OK	-	NO
A3	A	1	OK	-	NO
A1	A	1	OK	-	NO
A4	A	1	OK	-	NO
A9	A	1	OK	-	NO
A10	A	1	OK	-	NO
A14	A	1	OK	-	NO
A5	A	1	ERROR	The transaction e...	NO
B2	B	1	OK	-	NO
B0	B	1	OK	-	NO
B3	B	1	OK	-	NO
B7	B	1	OK	-	NO
B8	B	1	OK	-	NO

Form1

Type A

65

Type B

80

Isolation Level

READ UNCOMMITTED

Final Deadlock Report:

Deadlocks A: 0

Deadlocks B: 0

Tamam

	Thread ID	User Type	Step	Status	Execution Time	Deadlock
▶	SYSTEM	-	-	INFO	Simulation Starte...	-
	A19	A	1	OK	-	NO
	A63	A	1	OK	-	NO
	B0	B	1	OK	-	NO
	B1	B	1	OK	-	NO
	A64	A	1	OK	-	NO
	B7	B	1	OK	-	NO
	B12	B	1	OK	-	NO
	B13	B	1	OK	-	NO
	B5	B	1	OK	-	NO
	B10	B	1	OK	-	NO
	B11	B	1	OK	-	NO
	A4	A	1	ERROR	The transaction e...	NO
	B9	B	1	OK	-	NO

Start Simulation

Form1

Type A

99

Type B

99

Isolation Level

REPEATABLE READ

	Thread ID	User Type	Step	Status	Execution Time	Deadlock
▶	SYSTEM	-	-	INFO	Simulation Starte...	-
	B6	B	1	OK	-	NO
	B7	B	1	OK	-	NO
	B29	B	1	OK	-	NO
	B70	B	1	OK	-	NO
	B82	B	1	OK	-	NO
	A0	A	1	ERROR	Transaction (Pro...	YES
	A4	A	1	ERROR	Transaction (Pro...	YES
	A10	A	1	ERROR	Transaction (Pro...	YES
	A2	A	1	ERROR	Transaction (Pro...	YES
	A3	A	1	ERROR	Transaction (Pro...	YES
	A5	A	1	ERROR	Transaction (Pro...	YES
	A17	A	1	ERROR	Transaction (Pro...	YES
	A7	A	1	ERROR	Transaction (Pro...	YES

Start Simulation

Form1

Type A 99 **Type B** 99

Isolation Level REPEATABLE READ

	Thread ID	User Type	Step	Status	Execution Time	Deadlock
▶	SYSTEM	-	-	INFO	Simulation Starte...	-
	A19	A	1	OK	-	NO
	B87	B	1	OK	-	NO
	B88	B	1	OK	-	NO
	A7	A	1	ERROR	Transaction (Pro...	YES
	A7	A	2	OK	-	NO
	A1	A	1	ERROR	Transaction (Pro...	YES
*						

Start Simulation

Form1

Type A 95 **Type B** 98

Isolation Level READ COMMITTED

	Thread ID	User Type	Step	Status	Execution Time	Deadlock
	A40	A	TOTAL	TIME	2858792.8998 ms	-
	A91	A	98	ERROR	The transaction e...	NO
	A68	A	100	ERROR	The transaction e...	NO
	A68	A	TOTAL	TIME	2860492.0021 ms	-
	A66	A	100	ERROR	The transaction e...	NO
	A66	A	TOTAL	TIME	2861547.8459 ms	-
	A89	A	99	OK	-	NO
	A69	A	100	ERROR	The transaction e...	NO
	A69	A	TOTAL	TIME	2870488.2225 ms	-
	A63	A	97	ERROR	The transaction e...	NO
	A88	A	100	ERROR	The transaction e...	NO
	A88	A	TOTAL	TIME	2872626.3873 ms	-
	A67	A	99	ERROR	Yürütme Zaman ...	NO
	A76	A	100	ERROR	Yürütme Zaman ...	NO

Start Simulation

Final Deadlock Report:
Deadlocks A: 1237
Deadlocks B: 0

Tamam

Form1

Type A 50 Type B 66

Isolation Level READ COMMITTED

Thread ID	User Type	Step	Status	Execution Time	Deadlock
SYSTEM	-	-	INFO	Simulation State...	-
A7	A	1	OK	-	NO
A25	A	1	OK	-	NO
A29	A	1	OK	-	NO
A28	A	1	OK	-	NO
A47	A	1	OK	-	NO
A48	A	1	OK	-	NO
A46	A	1	OK	-	NO
B5	B	1	OK	-	NO
B2	B	1	OK	-	NO
B3	B	1	OK	-	NO
B4	B	1	OK	-	NO
B6	B	1	OK	-	NO
B12	B	1	OK	-	NO

Start Simulation

DatabaseSimulator

Diagnostic Tools

Diagnostics session: 40 seconds

Events

Process Memory (MB)

CPU (% of all processors)

Summary Events Memory Usage CPU Usage

Events

Memory Usage

CPU Usage

DatabaseSimulator

Process: [35012] DatabaseSimulator.exe

Database.cs Database.cs [Design]

DatabaseSimulator DatabaseSimulator Database

Execute/UpdateQuery/SqlCommand cmd, string beginStr, string er...

Form1

Type A 61 Type B 46

Isolation Level SERIALIZABLE

Thread ID	User Type	Step	Status	Execution Time	Deadlock
SYSTEM	-	-	INFO	Simulation State...	-
A29	A	1	OK	-	NO
A30	A	1	OK	-	NO
A32	A	1	OK	-	NO
B2	B	1	OK	-	NO
B3	B	1	OK	-	NO
B17	B	1	OK	-	NO
B18	B	1	OK	-	NO
B25	B	1	OK	-	NO
B24	B	1	OK	-	NO
B26	B	1	OK	-	NO
A15	A	1	ERROR	Transaction (Pro...	YES
A60	A	1	ERROR	Transaction (Pro...	YES
B0	B	1	ERROR	Transaction (Pro...	YES

Start Simulation

Diagnostic Tools

Diagnostics session: 28 seconds

Events

Process Memory (MB)

CPU (% of all processors)

Summary Events Memory Usage CPU Usage

Events

Memory Usage

CPU Usage

Form1

Type A

100

Type B

100

Isolation Level

READ UNCOMMITTED

	Thread ID	User Type	Step	Status	Execution Time	Deadlock
	A74	A	100	ERROR	The transaction e...	NO
	A74	A	TOTAL	TIME	2945006,9071 ms	-
	A80	A	100	ERROR	The transaction e...	NO
	A80	A	TOTAL	TIME	2945772,6739 ms	-
	A45	A	100	ERROR	The transaction e...	NO
	A45	A	TOTAL	TIME	2947939,0451 ms	-
	A66	A	100	ERROR	The transaction e...	NO
	A66	A	TOTAL	TIME	2949649,1983 ms	-
	A35	A	100	ERROR	The transaction e...	NO
	A35	A	TOTAL	TIME	2954097,1518 ms	-
	A82	A	100	ERROR	The transaction e...	NO
	A82	A	TOTAL	TIME	2957047,7195 ms	-
*						

Start Simulation

Form1

Type A

97

Type B

100

Isolation Level

SERIALIZABLE

	Thread ID	User Type	Step	Status	Execution Time	Deadlock
▶	SYSTEM	-	-	INFO	Simulation Starte...	-
	A57	A	1	OK	-	NO
	A85	A	1	OK	-	NO
	B20	B	1	OK	-	NO
	B21	B	1	OK	-	NO
	B22	B	1	OK	-	NO
	B44	B	1	OK	-	NO
	B57	B	1	OK	-	NO
	A0	A	1	ERROR	Transaction (Pro...	YES
	B3	B	1	ERROR	Transaction (Pro...	YES
	B0	B	1	ERROR	Transaction (Pro...	YES
	A2	A	1	ERROR	Transaction (Pro...	YES
	B1	B	1	ERROR	Transaction (Pro...	YES
	B11	B	1	ERROR	Transaction (Pro...	YES

Start Simulation

9. Source Code

```
using System.Collections.Generic;
using System.Data.SqlClient;
using System.Data;
using System.Threading;
using System.Windows.Forms;
using System;

namespace DatabaseSimulator
{
    public partial class Database : Form
    {
        private static int deadlockCountA = 0;
        private static int deadlockCountB = 0;
        private static readonly object deadlockLock = new object();

        public Database()
        {
            InitializeComponent();
            InitializeDataGridView();
            LoadIsolationLevels();
        }

        private void InitializeDataGridView()
        {
            dataGridViewLogs.Columns.Add("ThreadID", "Thread ID");
            dataGridViewLogs.Columns.Add("UserType", "User Type");
            dataGridViewLogs.Columns.Add("Step", "Step");
            dataGridViewLogs.Columns.Add("Status", "Status");
            dataGridViewLogs.Columns.Add("ExecutionTime", "Execution Time");
            dataGridViewLogs.Columns.Add("Deadlock", "Deadlock");
        }

        private void LoadIsolationLevels()
        {
            cmbIsolation.Items.Add("READ UNCOMMITTED");
            cmbIsolation.Items.Add("READ COMMITTED");
            cmbIsolation.Items.Add("REPEATABLE READ");
            cmbIsolation.Items.Add("SERIALIZABLE");
            cmbIsolation.SelectedIndex = 0;
        }

        private IsolationLevel GetIsolationLevel(string level)
        {
            switch (level)
            {
                case "READ UNCOMMITTED": return IsolationLevel.ReadUncommitted;
                case "READ COMMITTED": return IsolationLevel.ReadCommitted;
                case "REPEATABLE READ": return IsolationLevel.RepeatableRead;
                case "SERIALIZABLE": return IsolationLevel.Serializable;
            }
        }
    }
}
```

```

        default: return IsolationLevel.ReadCommitted;
    }
}

private void btnStartSimulation_Click(object sender, EventArgs e)
{
    deadlockCountA = 0;
    deadlockCountB = 0;

    int typeACount = (int)numericUpDownTypeA.Value;
    int typeBCount = (int)numericUpDownTypeB.Value;
    string isolationLevel = cmbIsolation.SelectedItem?.ToString();

    if (string.IsNullOrEmpty(isolationLevel))
    {
        MessageBox.Show("Please select an isolation level.");
        return;
    }

    dataGridViewLogs.Rows.Add("SYSTEM", "-", "-", "INFO", $"Simulation Started. Type A: {typeACount}, Type B: {typeBCount}, Isolation: {isolationLevel}", "-");

    StartSimulation(typeACount, typeBCount, isolationLevel);
}

private void StartSimulation(int typeA, int typeB, string isolationLevel)
{
    List<Thread> threads = new List<Thread>();

    for (int i = 0; i < typeA; i++)
    {
        int userId = i;
        Thread threadA = new Thread(() => SimulateTypeA(userId, isolationLevel));
        threadA.Start();
        threads.Add(threadA);
    }

    for (int i = 0; i < typeB; i++)
    {
        int userId = i;
        Thread threadB = new Thread(() => SimulateTypeB(userId, isolationLevel));
        threadB.Start();
        threads.Add(threadB);
    }

    Thread resultThread = new Thread(() =>
    {
        foreach (Thread t in threads)
        {
            t.Join();
        }
    })

```

```

        this.Invoke((MethodInvoker)delegate
        {
            ShowFinalDeadlockReport();
        });
    });

    resultThread.Start();
}

private void UpdateUI(Action updateAction)
{
    if (InvokeRequired)
    {
        this.Invoke(new Action(() => { updateAction(); }));
    }
    else
    {
        updateAction();
    }
}

private void SimulateTypeA(int id, string isolationLevel)
{
    string connectionString =
"Server=DEDEOGLU\\SQLEXPRESS;Database=AdventureWorks2022;Integrated
Security=True;Max Pool Size=200;";
    DateTime startTime = DateTime.Now;
    Random rand = new Random();

    for (int i = 0; i < 100; i++)
    {
        try
        {
            using (SqlConnection conn = new SqlConnection(connectionString))
            {
                conn.Open();
                SqlTransaction tran = conn.BeginTransaction(GetIsolationLevel(isolationLevel));

                SqlCommand cmd = new SqlCommand();
                cmd.Connection = conn;
                cmd.Transaction = tran;

                if (rand.NextDouble() < 0.5) ExecuteUpdateQuery(cmd, "20110101",
"20111231");
                if (rand.NextDouble() < 0.5) ExecuteUpdateQuery(cmd, "20120101",
"20121231");
                if (rand.NextDouble() < 0.5) ExecuteUpdateQuery(cmd, "20130101",
"20131231");
                if (rand.NextDouble() < 0.5) ExecuteUpdateQuery(cmd, "20140101",
"20141231");
                if (rand.NextDouble() < 0.5) ExecuteUpdateQuery(cmd, "20150101",
"20151231");
            }
        }
        catch { }
    }
}

```

```

        tran.Commit();
        conn.Close();
    }

    UpdateUI(() => {
        dataGridViewLogs.Rows.Add($"A{id}", "A", i + 1, "OK", "-", "NO");
    });

    Thread.Sleep(50);
}
catch (SqlException ex)
{
    bool isDeadlock = false;
    if (ex.Number == 1205)
    {
        IncrementDeadlockA();
        isDeadlock = true;
    }
    UpdateUI(() => {
        dataGridViewLogs.Rows.Add($"A{id}", "A", i + 1, "ERROR", ex.Message,
isDeadlock ? "YES" : "NO");
    });
}
}

    TimeSpan elapsed = DateTime.Now - startTime;
    UpdateUI(() => {
        dataGridViewLogs.Rows.Add($"A{id}", "A", "TOTAL", "TIME",
$"{{elapsed.TotalMilliseconds}} ms", "-");
    });
}

private void SimulateTypeB(int id, string isolationLevel)
{
    string connectionString =
"Server=DEDEOGLU\\SQLEXPRESS;Database=AdventureWorks2022;Integrated
Security=True;Max Pool Size=200;";
    DateTime startTime = DateTime.Now;
    Random rand = new Random();

    for (int i = 0; i < 100; i++)
    {
        Thread.Sleep(10);
        try
        {
            using (SqlConnection conn = new SqlConnection(connectionString))
            {
                conn.Open();
                SqlTransaction tran = conn.BeginTransaction(GetIsolationLevel(isolationLevel));

                SqlCommand cmd = new SqlCommand();

```

```

        cmd.Connection = conn;
        cmd.Transaction = tran;

        if (rand.NextDouble() < 0.5) ExecuteSelectQuery(cmd, "20110101",
"20111231");
        if (rand.NextDouble() < 0.5) ExecuteSelectQuery(cmd, "20120101",
"20121231");
        if (rand.NextDouble() < 0.5) ExecuteSelectQuery(cmd, "20130101",
"20131231");
        if (rand.NextDouble() < 0.5) ExecuteSelectQuery(cmd, "20140101",
"20141231");
        if (rand.NextDouble() < 0.5) ExecuteSelectQuery(cmd, "20150101",
"20151231");

        tran.Commit();
        conn.Close();
    }

    UpdateUI(() => {
        dataGridViewLogs.Rows.Add($"B{id}", "B", i + 1, "OK", "-", "NO");
    });

    Thread.Sleep(50);
}
catch (SqlException ex)
{
    bool isDeadlock = false;
    if (ex.Number == 1205)
    {
        IncrementDeadlockB();
        isDeadlock = true;
    }
    UpdateUI(() => {
        dataGridViewLogs.Rows.Add($"B{id}", "B", i + 1, "ERROR", ex.Message,
isDeadlock ? "YES" : "NO");
    });
}
}

    TimeSpan elapsed = DateTime.Now - startTime;
    UpdateUI(() => {
        dataGridViewLogs.Rows.Add($"B{id}", "B", "TOTAL", "TIME",
$"{{elapsed.TotalMilliseconds}} ms", "-");
    });
}

private void IncrementDeadlockA()
{
    lock (deadlockLock)
    {
        deadlockCountA++;
    }
}

```

```

    }

    private void IncrementDeadlockB()
    {
        lock (deadlockLock)
        {
            deadlockCountB++;
        }
    }

    private void ShowFinalDeadlockReport()
    {
        MessageBox.Show($"Final Deadlock Report:\nDeadlocks A: {deadlockCountA}\nDeadlocks B: {deadlockCountB}");
    }

    private void ExecuteUpdateQuery(SqlCommand cmd, string beginStr, string endStr)
    {
        cmd.Parameters.Clear();
        cmd.CommandText = @"
UPDATE Sales.SalesOrderDetail
SET UnitPrice = UnitPrice * 10.0 / 10.0
WHERE UnitPrice > 100
AND EXISTS (
    SELECT * FROM Sales.SalesOrderHeader
    WHERE SalesOrderHeader.SalesOrderID = SalesOrderDetail.SalesOrderID
    AND SalesOrderHeader.OrderDate BETWEEN @BeginDate AND @EndDate
    AND SalesOrderHeader.OnlineOrderFlag = 1)";

        DateTime begin = DateTime.ParseExact(beginStr, "yyyyMMdd", null);
        DateTime end = DateTime.ParseExact(endStr, "yyyyMMdd", null);
        cmd.Parameters.AddWithValue("@BeginDate", begin);
        cmd.Parameters.AddWithValue("@EndDate", end);
        cmd.ExecuteNonQuery();
    }

    private void ExecuteSelectQuery(SqlCommand cmd, string beginStr, string endStr)
    {
        cmd.Parameters.Clear();
        cmd.CommandText = @"
SELECT COUNT(*)
FROM Sales.SalesOrderDetail
WHERE UnitPrice > 100
AND EXISTS (
    SELECT * FROM Sales.SalesOrderHeader
    WHERE SalesOrderHeader.SalesOrderID = SalesOrderDetail.SalesOrderID
    AND SalesOrderHeader.OrderDate BETWEEN @BeginDate AND @EndDate
    AND SalesOrderHeader.OnlineOrderFlag = 1)";

        DateTime begin = DateTime.ParseExact(beginStr, "yyyyMMdd", null);
        DateTime end = DateTime.ParseExact(endStr, "yyyyMMdd", null);
        cmd.Parameters.AddWithValue("@BeginDate", begin);
    }

```

```
        cmd.Parameters.AddWithValue("@EndDate", end);  
        cmd.ExecuteScalar();  
    }  
}  
}
```